

Poisson 图像编辑实验报告

学号：PB23000270

姓名：李佩优

2026 年 3 月 22 日

1 算法原理

Poisson 图像编辑的核心思想是，在保持目标图像（背景）中待编辑区域边界像素值不变的前提下，通过求解一个具有 Dirichlet 边界条件的 Poisson 方程，来平滑地填充该区域内部。这样，源图像的梯度信息可以被“引导”到目标区域，从而实现无缝融合。

设目标图像为 f^* ，源图像为 g ，目标图像中待修改的区域为 Ω ，其边界为 $\partial\Omega$ 。我们的目标是找到一个新函数 f ，它在 Ω 内部由源图像的梯度场 $\mathbf{v}$ 引导，在边界上与目标图像保持一致。该问题可以表述为如下变分问题：

$$\min_f \iint_{\Omega} |\nabla f - \mathbf{v}|^2, \quad \text{s.t. } f|_{\partial\Omega} = f^*|_{\partial\Omega}$$

该变分问题等价于求解如下 Poisson 方程：

$$\Delta f = \nabla \cdot \mathbf{v} \quad \text{在 } \Omega \text{ 内部}, \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}$$

其中 Δ 是拉普拉斯算子。当取引导向量场 $\mathbf{v}$ 为源图像的梯度场 ∇g 时，即实现了论文中的“Importing gradients”无缝克隆。此时方程变为：

$$\Delta f = \Delta g \quad \text{在 } \Omega \text{ 内部}, \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}$$

1.1 离散化与线性方程组

为了在计算机上求解，需要将上述连续方程离散化。对于图像中的每个像素 $p = (x, y)$ ，其拉普拉斯算子可以离散化为：

$$\Delta f(p) \approx 4f(p) - f(x+1, y) - f(x-1, y) - f(x, y+1) - f(x, y-1)$$

因此，Poisson 方程在像素 p 处的离散形式为：

$$4f(p) - \sum_{q \in N(p)} f(q) = \sum_{q \in N(p)} (g(p) - g(q)) + \sum_{q \in N(p) \cap \partial\Omega} f^*(q)$$

其中 $N(p)$ 表示 p 的四个邻域（上下左右）。右侧第一项是源图像的梯度差异，第二项是边界贡献。

对于 Ω 内的每个像素，我们都可以列出这样一个方程，最终构成一个关于区域内所有像素颜色值的线性方程组 $\mathbf{Ax} = \mathbf{b}$ 。矩阵 $\mathbf{A}$ 是一个高度稀疏的对称正定矩阵，其非零元分布与像素的邻接关系一致。

2 实现细节

2.1 面向对象设计

为了实现 Poisson 融合算法，我设计了一个 ‘Seamless’ 类（继承自 ‘Cloner’），它封装了所有相关的计算和数据结构。该类提供了以下核心接口：

- `setSrcImage()` / `setTarImage()`: 设置源图像和目标图像。
- `setSelectedRegion()`: 设置用户选中的区域（像素坐标列表）。
- `presolve()`: 构建并预分解系数矩阵 $\mathbf{A}$ ，为实时求解做准备。
- `solve()`: 根据当前鼠标位置（即偏移量），构建右侧向量 $\mathbf{b}$ 并求解，返回融合后的图像。
- `correction()`: 当所选区域部分超出目标图像边界时，裁剪无效区域并更新矩阵。

2.2 稀疏矩阵的构建

系数矩阵 $\mathbf{A}$ 的维度为 $N \times N$ ，其中 N 是选中区域内的像素总数。在 ‘Seamless::buildMatrix()’ 函数中，我们遍历区域内的每个像素，并为其添加方程。由于每个像素最多有 5 个非零系数（自身和 4 个邻域），矩阵是高度稀疏的。我们使用 Eigen 库的 ‘Eigen::SparseMatrix<float>’ 类型和 ‘Eigen::Triplet<float>’ 列表来构建它。

代码实现如下：

Listing 1: 构建系数矩阵 (buildMatrix)

```
1 Eigen::SparseMatrix<float> Seamless::buildMatrix() {
2     std::vector<Eigen::Triplet<float>> triplets;
3     triplets.reserve(5 * nPixels);
4     for (size_t i = 0; i < pixel_indices_.size(); ++i) {
5         auto q = pixel_indices_[i];
6         int x = q.first, y = q.second;
7         // 仅处理选中的有效像素
8         if (src_selected_mask_ -> get_pixel(x, y)[0] < 2 ) continue;
```

```

9
10     int neighbor_count = 0;
11     // 检查四个方向的邻居
12     std::vector<std::pair<int,int>> neighbors = {{x+1, y}, {x-1,
13         y}, {x, y+1}, {x, y-1}};
14     for (const auto& nb : neighbors) {
15         if (src_selected_mask_->get_pixel(nb.first, nb.second)[0]
16             > 1) {
17             auto it = pixel_to_index_.find(nb);
18             triplets.emplace_back(i, it->second, -1.0f);
19             ++neighbor_count;
20         }
21     }
22     // 对角元系数是邻居数量（最多4个）
23     triplets.emplace_back(i, i, static_cast<float>(neighbor_count
24         ));
25 }
26 Eigen::SparseMatrix<float> A(nPixels, nPixels);
27 A.setFromTriplets(triplets.begin(), triplets.end());
A.makeCompressed();
return A;
}

```

2.3 右侧向量的构建

右侧向量 $\mathbf{b}$ 的值取决于引导场 $\mathbf{v}$ 和边界条件。在 ‘Seamless::buildRHS(int channel)’ 中，我们为每个 RGB 通道分别构建。根据公式，对于像素 p ，其对应的 b_p 是所有邻域的梯度差之和，加上边界上邻域的已知颜色值。

Listing 2: 构建右侧向量 (buildRHS)

```

1 Eigen::VectorXf Seamless::buildRHS(int channel) {
2     Eigen::VectorXf b(nPixels);
3     for (size_t i = 0; i < pixel_indices_.size(); ++i) {
4         auto q = pixel_indices_[i];
5         int x = q.first, y = q.second;
6         float gp = src_img_->get_pixel(x, y)[channel];
7         float sum = 0.0f;
8         // 检查四个方向
9         for (const auto& d : dirs) {
10             int nx = x + d.first, ny = y + d.second;

```

```

11         bool inside_mask = (src_selected_mask_ -> get_pixel(nx, ny)
12             [0] > 1);
13         // 内部邻居：累加梯度差异
14         if (inside_mask) {
15             float gq = src_img_ -> get_pixel(nx, ny)[channel];
16             sum += (gp - gq);
17         } else {
18             // 外部邻居（边界）：累加目标图像的颜色值
19             int tx = offset_x_ + nx - region_min_x_;
20             int ty = offset_y_ + ny - region_min_y_;
21             if (tx >= 0 && tx < tar_img_ -> width() && ty >= 0 &&
22                 ty < tar_img_ -> height()) {
23                 sum += tar_img_ -> get_pixel(tx, ty)[channel];
24             }
25         }
26         b(i) = sum;
27     }
28     return b;
29 }

```

2.4 实时求解与矩阵预分解

为实现实时编辑，我们利用了系数矩阵 $\mathbf{A}$ 在拖动过程中不变的性质。在 ‘Seamless::presolve()’ 中，我们构建 $\mathbf{A}$ 并调用求解器的 ‘compute()’ 方法进行矩阵分解。这样，在每一帧的 ‘solve()’ 中，只需重新构建 $\mathbf{b}$ 并调用 ‘solve()’ 方法即可，计算开销大大降低。

Listing 3: 预分解与求解

```

1 bool Seamless::presolve(){
2     if (!src_img_ || !tar_img_ || nPixels == 0) return true;
3     if (A_.rows() != nPixels) {
4         A_ = buildMatrix();
5         solver_.compute(A_);
6         return solver_.info() != Eigen::Success;
7     }
8     return false;
9 }
10
11 std::shared_ptr<Image> Seamless::solve(){
12     // ... 偏移量检查 ...

```

```

13     for (int c = 0; c < 3; ++c) {
14         Eigen::VectorXf b = buildRHS(c);
15         result_[c] = solver_.solve(b); // 利用预分解快速求解
16         if (solver_.info() != Eigen::Success) return nullptr;
17     }
18     applyResultToImage();
19     return tar_img_;
20 }

```

2.5 不规则区域的支持

为了支持多边形等不规则区域的选择，我们在 *SourceImageWidget* 和 *Shape* 类中进行了扩展。多边形 *Polygon* 类实现了扫描线填充算法，通过 *getInteriorPixels()* 方法返回其内部所有像素的坐标。该算法逐行扫描，计算每条扫描线与多边形边的交点，然后配对填充，从而得到完整的区域掩码。这使得 Poisson 融合可以应用于任意形状的区域。

3 实验结果与分析

3.1 测试设置

实验使用了框架提供的标准测试图像。源图像为一张带有老虎图案的图片，目标图像为一张风景图片。用户在多边形模式下选中老虎区域，并将其移动到目标图像的草地上进行融合。

3.2 矩形区域融合效果

首先测试了矩形区域的融合。如图??所示，简单粘贴（Paste）会导致明显的边界和颜色不匹配。而 Poisson 融合（Seamless）的结果则自然地融入了背景，老虎的轮廓与草地实现了无缝过渡，验证了算法的有效性。

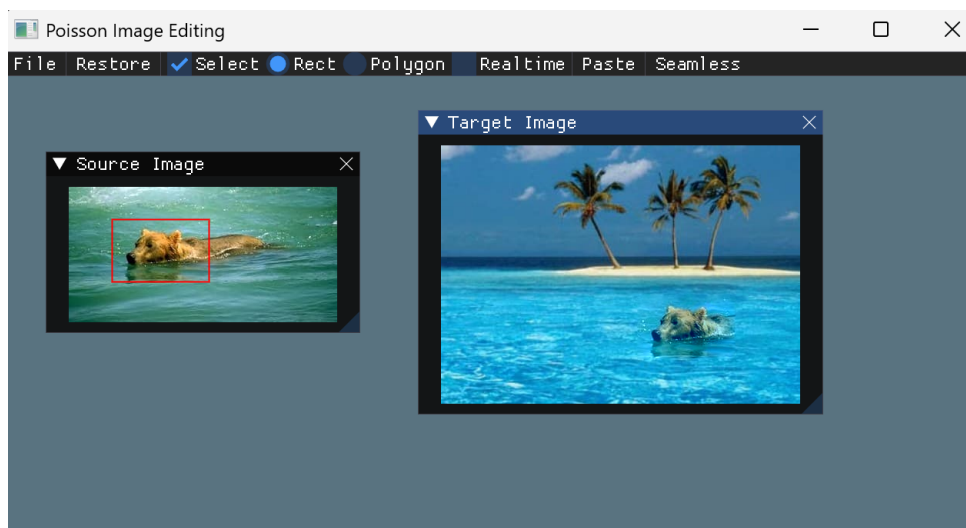


图 1: 矩形区域融合效果对比 (左: 源区域, 中: Paste, 右: Seamless)

3.3 多边形区域融合效果

接下来测试了多边形区域的融合。如图??所示, 用户可以选择任意形状的区域 (如老虎的身体轮廓)。Poisson 融合结果在复杂边界处依然保持平滑, 没有出现明显的接缝或伪影, 证明了不规则区域处理方法的有效性。

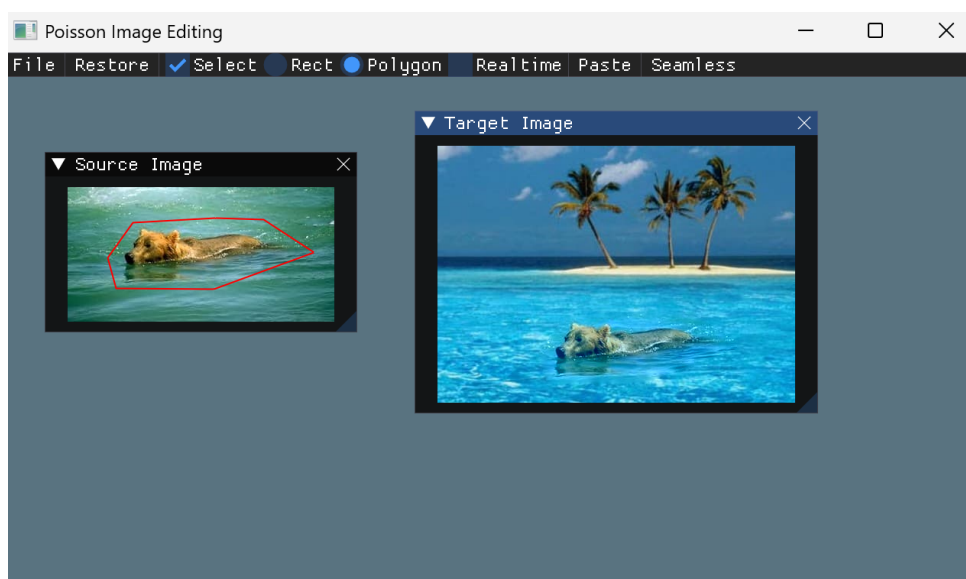


图 2: 多边形区域融合效果

3.4 实时编辑性能

在 Release 模式下运行程序, 并开启“Realtime”选项。测试表明, 对于包含约 10,000 个像素的选中区域, 整个融合过程 (包括构建 $\mathbf{b}$ 和解线性系统) 可以在毫秒级完成, 使

得鼠标拖动时能流畅地更新结果，达到了实时交互的要求。这得益于 Eigen 库高效的稀疏求解器和矩阵预分解技术。

4 讨论与改进

4.1 边界裁剪处理

当所选区域的一部分移出目标图像边界时，融合问题变为混合边值问题，失去了完整的 Dirichlet 边界条件。为此，我在 ‘Seamless::correction()’ 中实现了边界裁剪逻辑：检测超出边界的像素并将其从区域中移除，然后重新构建矩阵并预分解。这保证了求解过程的稳定性。

4.2 求解器选择

实验中选择了 ‘Eigen::SparseLU’ 作为求解器。由于矩阵 $\mathbf{A}$ 是对称正定的，理论上可以使用 ‘SimplicialLDLT’ 等更高效的求解器。但在我的测试中，‘SparseLU’ 提供了足够的稳定性，并且预分解速度较快，满足实时性要求。未来可以进一步优化求解器的选择以提升性能。

4.3 混合梯度的扩展

根据论文 [1]，混合梯度方法可以更好地保留源图像中的纹理细节，同时避免颜色偏差。其实现仅需修改 ‘buildRHS()’ 中的梯度计算部分： $\sum_{q \in N(p)} (g(p) - g(q))$ 改为 $\sum_{q \in N(p)} \max(|g(p) - g(q)|, |f^*(p) - f^*(q)|)$ 。在代码中，这需要访问目标图像在对应位置的像素值。由于时间原因，我暂未实现此扩展，但它是一个有价值的改进方向。

5 结论

本实验成功实现了基于 Poisson 方程的无缝图像融合算法。通过离散化 Poisson 方程并构造大型稀疏线性方程组，利用 Eigen 库进行高效的矩阵分解和求解，实现了高质量的图像融合效果。同时，通过预分解技术，达到了鼠标拖拽时的实时编辑性能。此外，还扩展了多边形区域的选择功能，使得交互更加灵活。实验结果表明，该方法能有效消除融合图像中的边界瑕疵，生成自然、无缝的合成图像。

参考文献

- [1] Pérez, P., Gangnet, M., & Blake, A. (2003). Poisson image editing. *ACM Transactions on Graphics (TOG)*, 22(3), 313-318.

- [2] Guennebaud, G., Jacob, B., & others. (2010). Eigen v3. <http://eigen.tuxfamily.org>